LAB TASK: SOURCE CODE:

```
;program to print capital latters from(A-Z) in same line and
;small letter from (z-a).every letter on next line.
.model small
.stack 100h
.data
   num db ?
.code
main proc


   mov cx,26
   mov bl,122
   mov num,65

   L1:
      mov ah,02h
      mov dl,num
      int 21h

      add num,1
    loop L1
    mov ah,02h
    mov dl,0dh
    int 21h

    mov dl,0ah
    int 21h

    mov cx,26
      L2:

     mov ah,02h

      mov dl,bl
      int 21h
```



```
        mov ah,02h
        mov dl,0dh
        int 21h

        mov dl,0ah
        int 21h

        dec bl
        loop L2

        mov ah,4ch
        int 21h

 main endp
end main
```

TASK: SOURCE CODE:

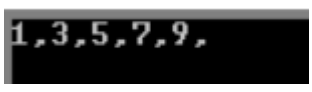
```
;program to print odd numbers from 0 to 9
.model small
.stack 100
.code
main proc

    mov cx,5
    mov bl,49

    L1:
        mov ah,02h
        mov dl,bl
        int 21h

        mov dl,','
        int 21h

        add bl,2
```



```
        loop L1

        mov ah,4ch
        int 21h
main endp
end main
```

TASK4:

```
.model small
.stack 100
.code
main proc
    mov cl,4
    mov bl,50

    L1:
        mov ah,02h
        mov dl,bl
        int 21h

        mov dl,','
        int 21h

        add bl,2

        loop L1

        mov ah,4ch
        int 21h
main endp
end main
```



OUTPUT

TASK5: SOURCE CODE:

```
;program to print even numbers from 0 to 9
.model small
```

```
.stack 100
.data
   num db ?
.code
main proc
   sub cx,cx
   mov cl,5
   mov num,48

   L1:
      mov ah,02h
      mov dl,num
      int 21h

      mov dl,','
      int 21h

      add num,2

      loop L1

      mov ah,4ch
      int 21h
main endp
end main
```

OUTPUT:



TASK1: SOURCE CODE:

```
;program to print a letter on the center of the console using
;video interrupt..
.model small
.stack 100h
.data
    num db ?
.code
main proc


    mov ah,02h

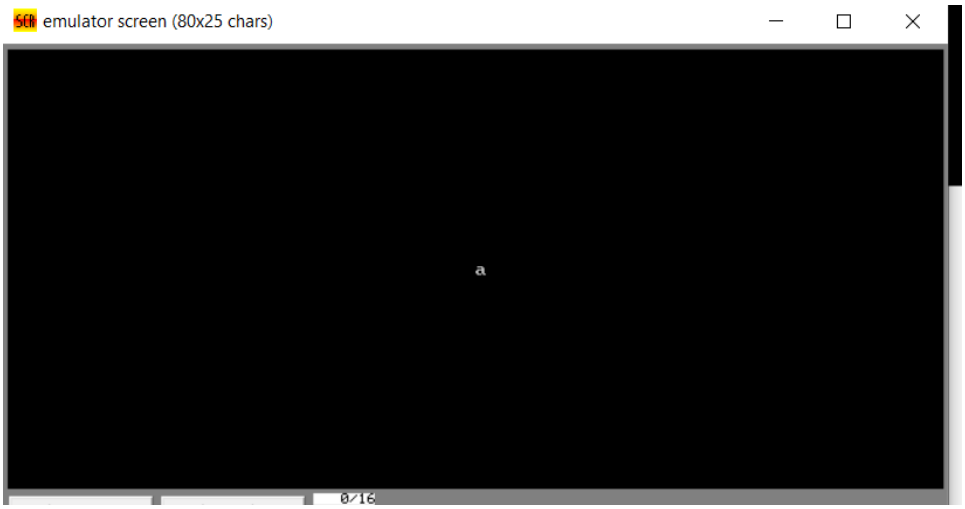
    mov dl,40
    mov dh,12
    mov bx,0
    int 10h

    mov dl,'a';
    int 21h


      mov ah,4ch
      int 21h
```

```
 main endp
end main
```

OUTPUT:



TASK2: SOURCE CODE:

```
;program to mov a string right and left on the console using
;video interrupt..
.model small
.stack 100h
.data
    msg db " SUKKUR IBA $"
    i db ?
.code
main proc
   mov ax,@data
   mov ds,ax
```

```
mov cx,69
mov i,0 ;row

l1:
 mov ah,02h

 mov dl,i
 mov dh,0
 mov bx,0
 int 10h

 mov ah,09h
 lea dx,msg

 inc i

 int 21h

 loop l1
 mov cx,70
 l2:
 mov ah,02h

 mov dl,i
 mov dh,0
 mov bx,0
 int 10h

 mov ah,09h
 lea dx,msg

 dec i

 int 21h

 loop l2
```

```
    mov ah,4ch
    int 21h

 main endp
end main
```

OUTPUT:



TASK2:

SOURCE CODE:

```
;simple number hitting game
.model small
.stack 100h
.data
    msg db "Enter a number to hit (0-9): $"
    arrow db " ====>$"
    i db ?
    num db ?
    j db ?
.code
main proc
   mov ax,@data
   mov ds,ax

   mov ah,09h
   lea dx,msg
   int 21h
```

```
mov i,1 ;row
mov cx,10
mov num,48

l1:
    mov ah,02h
    mov dl,70   ;col
    mov dh,i    ;row
    mov bx,0    ;page
    int 10h

  mov ah,02h
  mov dl,num
  int 21h

  mov dl,0dh
  int 21h

  mov dl,0ah
  int 21h

  inc num
  inc i
  loop l1

mov ah,01
int 21h

mov j,al
sub j,47

mov cx,66
mov i,0

l2:
 mov ah,02h

 mov dl,i
```

```
    mov dh,j
    mov bx,0
    int 10h

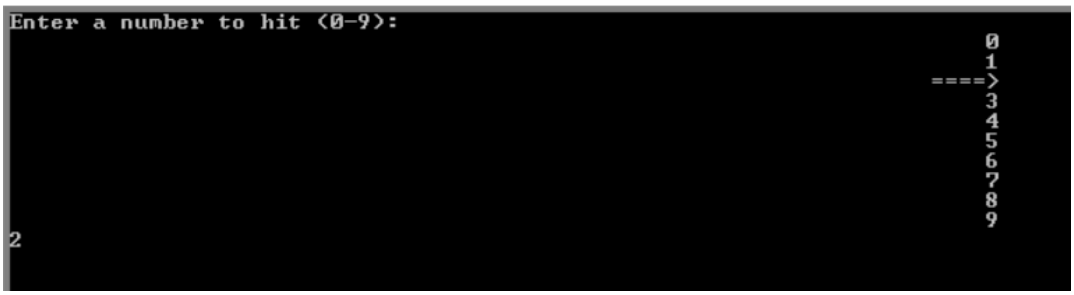
    mov ah,09h
    lea dx,arrow
    int 21h

    inc i
    loop l2

      mov ah,4ch
      int 21h

 main endp
end main
```

OUTPUT:



CONTROL FLOW STRUCTURES & CASE STRUCTURE

TASK1: Write a code to read two capital letters and display them in alphabetical order.

SOURCE CODE:

```
.model small
.stack 100h
.data
    msg1 db "Enter 2 capital letters : $"
    msg2 db "Invalid letter! Try again : $"
    val1 db ?
    val2 db ?
.code
main proc
      mov ax,@data
      mov ds,ax

      mov ah,09
      lea dx,msg1
      int 21h

     mov cx,2

   l1:
        mov ah,01
        int 21h

        cmp al,'A'
        jge b1
        mov ah,02h
         mov dl,0ah
        int 21h

        mov dl,0dh
```

```
  int 21h
  mov ah,09
  lea dx,msg2
  int 21h
  jmp l1:

  b1:
    cmp al,'Z'
    jle b2
    mov ah,02h
      mov dl,0ah
      int 21h

      mov dl,0dh
      int 21h
    mov ah,09
    lea dx,msg2
    int 21h
    jmp l1
    b2:
    cmp cx,2
    je b3
    jmp b4
    b3:
    mov bl,al
    jmp o
    b4:
    mov bh,al

    o:
loop l1:

mov ah,02h
mov dl,0ah
int 21h
```

```
    mov dl,0dh
    int 21h

    cmp bl,bh
    jle b5
    jmp else

    b5:
    mov ah,02h
    mov dl,bl
    int 21h

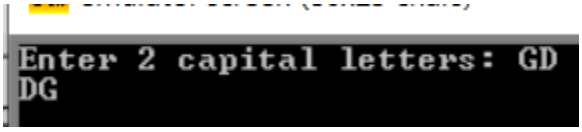
    mov ah,02h
    mov dl,bh
    int 21h
    jmp exit
    else:
    mov ah,02h
    mov dl,bh
    int 21h


    mov ah,02h
    mov dl,bl
    int 21h

    exit:
    mov ah,4ch
    int 21h

end main
main endp
```

OUTPUT:



TASK2: Write a code to read a number from (3 to 5), if a number in range display number otherwise display “Invalid number” .

SOURCE CODE:

```
.model small
.stack 100h
.data
   msg1 db "Enter a number from (3 to 5) : $"
   msg2 db "Invalid number! $"
   val1 db ?
   val2 db ?
.code
main proc
     mov ax,@data
     mov ds,ax

     mov ah,09
     lea dx,msg1
     int 21h

     mov ah,01
     int 21h

     mov bl,al

     mov ah,02h
     mov dl,0ah
     int 21h

     mov dl,0dh
```

```
    int 21h

    cmp bl,'3'
    jge b1
    jmp else

    b1:
      cmp bl,'5'
      jle b2
      jmp else

      b2:
      mov ah,02
      mov dl,bl
      int 21h
      jmp exit

    else:
       mov ah,09
       lea dx,msg2
       int 21h

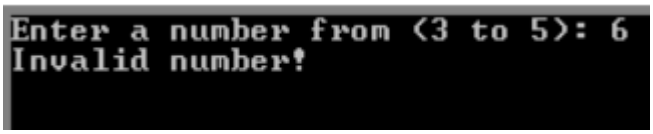



    exit:
    mov ah,4ch
    int 21h

end main
main endp
```

OUTPUT:





TASK3: Write a code to read a number from (0-4) and display in words.

SOURCE CODE:

```
.model small
.stack 100h
.data
    msg1 db "Enter a number from (0 to 4) : $"
    zero db "Zero$"
    one db "One$"
    two db "Two$"
    three db "Three$"
    four db "Four$"

.code
main proc
      mov ax,@data
      mov ds,ax

      mov ah,09
      lea dx,msg1
      int 21h
```

```
mov ah,01
int 21h

mov bl,al
;sub bl,48

mov ah,02h
mov dl,0ah
int 21h

mov dl,0dh
int 21h

cmp bl,'0'
je B_ZERO

cmp bl,'1'
je B_ONE

cmp bl,'2'
je B_TWO

cmp bl,'3'
je B_THREE

cmp bl,'4'
je B_FOUR

jmp exit

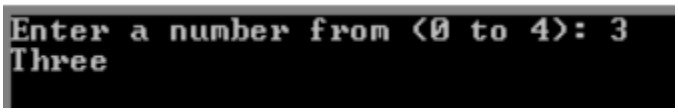
B_ZERO:
  mov ah,09
  lea dx,zero
  int 21h
  jmp exit
```

```
    B_ONE:
      mov ah,09
      lea dx,one
      int 21h
      jmp exit
    B_TWO:
      mov ah,09
      lea dx,two
      int 21h
      jmp exit
    B_THREE:
      mov ah,09
      lea dx,three
      int 21h
      jmp exit
    B_FOUR:
      mov ah,09
      lea dx,four
      int 21h



    exit:
    mov ah,4ch
    int 21h
main endp
end main
```

OUTPUT:



TASK4: Use a case structure to code the following

Read a character

If a character is ‘a’ display linefeed.

If a character is ‘b’ display Carriage return

If pressed another character return to dos.

SOURCE CODE:

```
.model small
.stack 100h
.data
   msg1 db "Enter a character : $"
   cr db "0dh$"
   lf db "0ah$"
.code
main proc
     mov ax,@data
     mov ds,ax

     mov ah,09
     lea dx,msg1
     int 21h
```

```
mov ah,01
int 21h

mov bl,al

mov ah,02h
mov dl,0ah
int 21h

mov dl,0dh
int 21h

cmp bl,'a'
je b1

cmp bl,'b'
je b2

jmp exit

b1:
  mov ah,09
  lea dx,lf
  int 21h
  jmp exit
b2:
  mov ah,09
  lea dx,cr
  int 21h

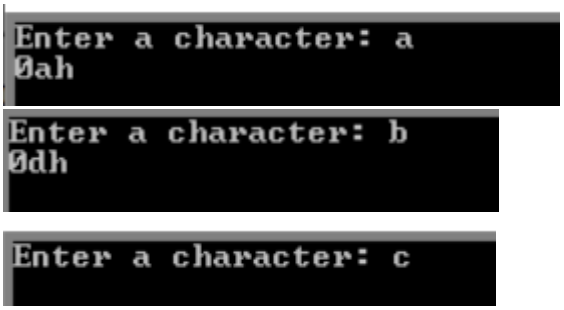



exit:
mov ah,4ch
```

```
    int 21h

end main
main endp
```

OUTPUT:



TASK1: Read an input and recognize the vowel letters only.

SOURCE CODE:

```
.model small
.stack 100h
.data
   str db  "Charachter is vowel$"
   str1 db  "Charachter is not vowel$"
.code
main proc

    mov ax,@data
    mov ds,ax

    mov ah,01
    int 21h

    mov bl,al

    mov ah,02h
```

```
mov dl,0ah
int 21h
mov dl,0dh
int 21h

cmp bl,'a'
je vowel

cmp bl,'e'
je vowel

cmp bl,'i'
je vowel

cmp bl,'o'
je vowel

cmp bl,'u'
je vowel

cmp bl,'A'
je vowel

cmp bl,'E'
je vowel

cmp bl,'I'
je vowel

cmp bl,'O'
je vowel

cmp bl,'U'
je vowel

jmp nonVowel
```

```
    vowel:
    mov ah,09
    lea dx,str
    int 21h

    jmp exit

    nonVowel:
    mov ah,09
    lea dx,str1
    int 21h

    exit:

    mov ah,4ch
    int 21h

end main
main endp
```

OUTPUT:





TASK2: Read an input and recognize the numbers only

SOURCE CODE:

```
.model small
.stack 100h
```

```
.data
    msg1 db "you entered a valid number$"
    msg2 db "you entered an Invalid number$"
.code
main proc
    mov ax,@data
    mov ds,ax

    mov ah,01
    int 21h

    mov bl,al

    mov ah,02
    mov dl,0ah
    int 21h

    mov dl,0dh
    int 21h


    cmp bl,'0'
    jge b1
    jmp b3
        b1:
        cmp bl,'9'
        jle b2
        jmp b3

        b2:
        mov ah,09
        lea dx,msg1
        int 21h
        jmp exit

        b3:
```

```
        mov ah,09
        lea dx,msg2
        int 21h

        exit:

        mov ah,4ch
        int 21h

end main
main endp
```

OUTPUT:

